

CSC 6303 Final Exam Spring 1 2026

Answer of Question# 1

Algol introduced the foundations of structured, high-level procedural programming, but C significantly transformed these ideas into a practical, hardware-oriented language suitable for real-world systems. The most important technical changes C introduced were:

C added **direct low-level memory access** through pointers (e.g., `&a` to obtain an address), enabling programmers to work closely with hardware—something Algol never supported. This made C suitable for operating systems, drivers, and embedded systems.

C also introduced a **far more compact and efficient syntax**, such as shortened keywords ("`int`" instead of Algol's "`Integer`") and condensed expressions like the ternary operator `a = (b < c) ? b : c;`. These features dramatically reduced verbosity and increased development speed.

Another major shift was C's **emphasis on performance and ergonomics**. C deliberately traded some safety for speed, allowing developers fine-grained control over memory, data layout, and execution. This contrasts with Algol's academic design philosophy, which focused on clean paradigms rather than runtime efficiency.

Together, these innovations made C not just an evolution of Algol, but a practical systems language whose influence shaped nearly all modern procedural languages—including C++, Java, Python, Go, and Swift.

Answer of Question# 2

2) Convert from Python to C++

Convert the code below from Python to C++ assuming the use of streams *cin* and *cout* for input and output.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
using namespace std;

char random_comp_move() {
    double u = static_cast<double>(rand()) / (RAND_MAX + 1.0);
    if (u < 1.0 / 3.0)    return 'r';
```

```

    else if (u < 2.0 / 3.0) return 's';
    else          return 'p';
}

void RSP() {
    string guess;

    while (true) {
        cout << "Enter (r)ock, (s)cissors, or (p)aper: ";
        cin >> guess;

        if (guess != "r" && guess != "s" && guess != "p") {
            cout << "Only 'r', 's', or 'p' are valid inputs! Please try again." << endl;
        } else {
            break;
        }
    }

    char comp = random_comp_move();

    if (guess[0] == comp) {
        cout << "It is a tie!" << endl;
    } else if ( (guess[0] == 'r' && comp == 'p') ||
                (guess[0] == 'p' && comp == 's') ||
                (guess[0] == 's' && comp == 'r') ) {
        cout << "Sorry, you lost as I had " << comp << endl;
    } else {
        cout << "Congrats, you won as I had " << comp << endl;
    }
}

int main() {
    srand(static_cast<unsigned int>(time(nullptr)));
    RSP();
    return 0;
}

```

Answer of Question# 3

3) Convert Python to Go

```

func letter(score int) string {

```

```

var grade string

if score >= 90 {
    grade = "A"
} else if score >= 80 {
    grade = "B"
} else if score >= 70 {
    grade = "C"
} else {
    grade = "F"
}

if score%10 >= 7 && grade != "A" && grade != "F" {
    grade += "+"
} else if score%10 <= 3 && grade != "F" {
    grade += "-"
}

return grade
}

```

Answer of Question# 4

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Button and Style</title>
    <style>
        h2 { color: red; }
        body { background-color: beige; color: grey; }
    </style>
</head>
<body>
    <h2>Click to change</h2>
    <p>Color and other properties can be changed using a JavaScript function.</p>

    <!-- Target element whose color and text will change -->
    <h3 id="change_js" style="color:green">This text is Green</h3>

    <script>
        // Change both color and visible text as required by the examples
        function changeColor(code) {
            const el = document.getElementById("change_js");
            if (code === 1) {

```

```

    el.style.color = "black";
    el.innerText = "This text is Black";
} else if (code === 2) {
    el.style.color = "blue";
    el.innerText = "This text is Blue";
} else {
    el.style.color = "green";
    el.innerText = "This text is Green";
}
}
</script>

```

```

<!-- Buttons keep the same behavior, just call the updated function -->
<button type="button" onclick="changeColor(1)">Black</button>
<button type="button" onclick="changeColor(2)">Blue</button>
<button type="button" onclick="changeColor(3)">Green</button>
</body>
</html>

```

Answer of Question# 5

The script checks whether the entered name corresponds to an **existing directory** using `if [ -d "$namedir" ]` and only creates a new directory if this condition is false.

The folder contents include an existing **directory** named `my_files` and an existing **file** named `letter.py`.

A) User enters: `my_folder`

- `my_folder` **does not exist** in `/Users/pf/test`.
- `-d "my_folder"` → **false**.
- Script runs: `mkdir my_folder` → a **new directory is created**.
- Script prints:
 "Folder my_folder created!"

Result: New folder successfully created.

B) User enters: `my_files`

- `my_files` **already exists** and is a **directory**.
- `-d "my_files"` → **true**.
- The script **does not** execute `mkdir`.
- Script prints:
 "Nothing to do, my_files exists already"

Result: No folder created; correct "exists already" message.

C) User enters: `letter.py`

- `letter.py` exists, but it is a **file**, not a directory.
- `-d "letter.py" → false` (because `-d` checks directories only).
- Script runs: `mkdir letter.py`
 - This **fails** because a file with that name already exists.
 - Terminal prints an error from `mkdir` (`stderr`):
"mkdir: letter.py: File exists"
- After the failed `mkdir`, the script still prints:
"Folder letter.py created!"

Result: User sees a `mkdir` error **and** a misleading success message.

Final Summary

Input	What Happens	Final Output
A) <code>my_folder</code>	Directory does not exist → created	"Folder my_folder created!"
B) <code>my_files</code>	Directory exists → no creation	"Nothing to do, my_files exists already"
C) <code>letter.py</code>	File exists → <code>mkdir</code> error + script still prints success	Error + "Folder letter.py created!"

Answer of Question# 6

Unix-like OS Item Permissions

In Unix-like systems, every file system item is represented by a **10-character permission descriptor**, structured as:
`[type][owner permissions][group permissions][others permissions]`

Applying the Required Permissions

The problem specifies:

1. **Readable by all users**
2. **Executable only by the group members**
3. **Writable only by the item owner**

We apply each rule to the Owner, Group, and Others permission fields.

Owner Permissions (3 characters)

Requirement: *written by the item owner alone and readable by all users.*

- Read (r): **yes**
- Write (w): **yes** — because only the owner may write
- Execute (x): **no** — not required and not allowed by the problem

So, the owner triplet is:

`rw-`

Group Permissions (3 characters)

Requirement:

- *read by all users* → group has **read**
- *executed only by users belonging to the group* → group gets **execute**
- *not writable* → group does **not** get write

So, the group triplet becomes:

`r-x`

Others Permissions (3 characters)

Requirement:

- All users may **read** → others get **r**
- Others may **not execute** → no **x**
- Others may **not write**

Thus, the others triplet is:

`r--`

Final Combined 10-Character Permission Descriptor

Combine:

- d (directory)
- rw- (owner)
- r-x (group)
- r-- (others)

Result:

drw-r-xr--

Final Answer :

drw-r-xr--

This descriptor fully satisfies all the conditions:

- The item is a **directory** (d).
- The **owner** can read and write.
- The **group** can read and execute.
- **All users** can read.
- No unintended write or execute permissions are granted.

Answer of Question# 7

A **Virtual Machine app** is the actual **software program** i install on my computer—such as VirtualBox, Parallels, VMware, or cloud-based tools like Google Colab. Its job is to act like a “manager” or “translator” between your real computer hardware and any virtual systems you want to run. In other words, the VM app is the tool that creates, controls, and runs virtual machines by simulating the hardware they need.

A **Virtual Machine**, on the other hand, is the **simulated computer** created *inside* that app. It behaves like a real independent computer with its own operating system (like Linux, Windows, or macOS), its own virtual CPU, memory, disk, and settings. You can install software inside it, change settings, or even break it—without affecting your actual physical computer. Essentially, the VM is the “computer inside a computer,” while the VM app is the software that makes this possible.

Answer of Question# 8

For tasks inside an algorithm to be split into multiple concurrent threads, they must be **independent enough** so that each task can run without constantly waiting for the others. In other words, the tasks need to be designed so

that their work can happen in **any order**, or even at the **same time**, without breaking the correctness of the final result. This requirement comes directly from the concept of **concurrency**, where tasks should not rely on each other's intermediate steps to move forward.

Your course explains this using the matrix-multiplication example: each element of the result matrix can be computed separately because it does not depend on the results of other elements; therefore each element-calculation can become its own concurrent thread. When tasks behave this way—no shared dependency, no need to wait, and no risk of interfering with each other—the operating system or programming language runtime can safely run them in parallel on different CPU cores. If tasks share data, then extra precautions like synchronization are needed, but the fundamental requirement remains: **the algorithm must contain parts that can be performed independently of the order in which they finish.**